

Lecture 5: Journaling File Systems

- File-system operations often modify multiple metadata structures.
- A crash between these updates can leave the file system inconsistent.
- Journaling records intended changes so the system can recover after a crash.

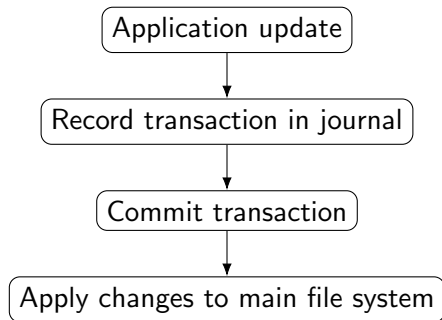
Learning Objectives

- Explain crash consistency and partial-update problems.
- Describe the structure and purpose of a journal.
- Distinguish metadata, ordered and full-data journaling.
- Explain journal commit and replay.
- Analyze journaling overhead.

Crash Consistency Problem

- Suppose creating a file requires changes to:
 - ① directory entry
 - ② inode/file metadata
 - ③ free-space information
- If power fails after only some writes reach storage, the structures may disagree.
- Example: a directory refers to a file whose metadata was never completely written.

The Basic Journaling Idea



Goal: after a crash, the journal provides enough information to recover to a consistent state.

- A transaction groups related updates.
- Typical conceptual sequence:
 - ① write transaction records to the journal
 - ② write a commit record
 - ③ later checkpoint/apply the changes to their final locations
- Recovery examines whether transactions were committed.

- Journal records metadata changes, such as:
 - inode updates
 - directory updates
 - allocation metadata
- File data may be written directly to its final location.
- Less journal traffic than journaling every data block.

Ordered Journaling

- Metadata updates are journaled.
- Data blocks are written to their final locations before the related metadata transaction is committed.
- Helps avoid metadata pointing to stale/uninitialized data.
- Balances consistency and journal overhead.

Full Data Journaling

- Both data and metadata are recorded in the journal.
- Provides strong ordering/recovery semantics.
- Generates more I/O and journal traffic.
- Useful when stronger consistency guarantees justify the cost.

Crash Recovery / Journal Replay

- 1 System restarts after a crash.
- 2 File system locates the journal.
- 3 Committed transactions are identified.
- 4 Required operations are replayed.
- 5 Incomplete/uncommitted transactions are ignored or rolled back according to the design.

Result: the file system can often become consistent without scanning the entire storage volume.

Journaling Trade-offs

Benefit	Cost
Fast recovery	Extra writes
Crash consistency	Journal storage
Reduced full-disk checking	Potential write latency
Predictable recovery behavior	Possible write amplification

- **Galvin, 10th Ed.:** Chapter 15 – File-System Internals, including recovery, journaling and soft updates.
- **Stallings, 9th Ed.:** Chapter 18 – Unix/Linux file management and file-system implementation material. Detailed journaling is not a major standalone heading in the chapter TOC, so use Stallings as supporting implementation reading.

Case Study Question

Problem

A crash occurs after a directory entry has reached disk, but the corresponding inode update has not. What inconsistency can result?

- What metadata should be protected?
- How would a journal help?
- Would metadata-only journaling be sufficient?

Lecture 6: Log-Structured File Systems

- A log-structured file system (LFS) treats storage as a large append-only log.
- Updates are accumulated and written sequentially.
- The design aims to exploit efficient sequential writes and reduce small random-write overhead.

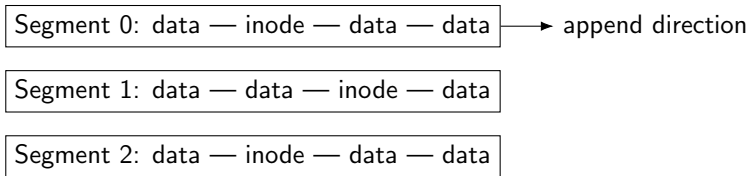
Learning Objectives

- Explain the motivation behind log-structured file systems.
- Describe segments, checkpoints and inode maps.
- Explain how old versions of blocks become obsolete.
- Describe segment cleaning/garbage collection.
- Analyze write amplification and performance trade-offs.

Motivation for LFS

- Traditional workloads may generate many small writes.
- Random writes can be expensive, especially on older disk technology.
- LFS collects updates and writes them sequentially.
many small updates → buffer/aggregate → large sequential write

Basic LFS Organization



- Storage is divided into large segments.
- New versions are appended rather than overwriting old locations immediately.

What Happens During an Update?

- 1 File block A currently exists at location X.
- 2 Application modifies A.
- 3 New version A' is appended at a new location.
- 4 Metadata is updated to point to A'.
- 5 Old A becomes obsolete.

old A $\rightarrow$ obsolete new A' $\rightarrow$ current

Inode Map

- Inodes may move as updates are appended.
- The system therefore needs a way to locate the current inode for a file.
- An **inode map** translates inode numbers to the current physical location of the inode.
- The map itself must be updated and recovered safely.

Checkpoint Region

- A checkpoint records enough information to restart the file system after a crash.
- It provides a known starting point for locating current metadata.
- Periodic checkpoints reduce the amount of information that recovery must reconstruct.

checkpoint → metadata locations → current file state

Why Is Cleaning Required?

- Append-only updates create obsolete blocks.
- Segments eventually contain a mixture of valid and invalid blocks.
- Free space is recovered by identifying valid blocks, copying them elsewhere and reclaiming the segment.

[valid] [invalid] [valid] [invalid] [invalid]

→ [valid] [valid] + reclaimed segment

Segment Cleaning

- 1 Select segments for cleaning.
- 2 Identify valid blocks.
- 3 Copy valid blocks to a new segment.
- 4 Update metadata references.
- 5 Mark old segments as free.

Challenge: cleaning itself creates additional writes.

- **User data written:** data requested by applications.
- **Physical data written:** user writes + metadata + cleaning copies + other system writes.

$$\text{Write Amplification} = \frac{\text{physical bytes written}}{\text{application bytes written}}$$

High cleaning overhead can reduce the benefit of sequential writing.

LFS Advantages and Limitations

Advantages

- Sequential write pattern
- Good small-write performance
- Natural versioning-like behavior
- Efficient use of write bandwidth in suitable workloads

Limitations

- Segment cleaning
- Write amplification
- Metadata complexity
- Performance depends strongly on workload

- **Galvin, 10th Ed.:** Chapter 15 – File-System Internals; Log-Structured File Systems and Recovery.
- **Stallings, 9th Ed.:** Chapter 18 – File Management, with supporting material on file organization, secondary storage and Unix/Linux implementation. LFS is not a separate major heading in the chapter TOC.

Lecture Summary

- LFS converts many updates into sequential log writes.
- Segments hold appended versions of data and metadata.
- Inode maps and checkpoints help locate the current file state.
- Cleaning reclaims space but introduces additional write traffic.